Package 'BioLogical'

November 18, 2025

Type Package

Title a universal logical-based analysis framework of biological complex systems

Version 0.1.19

Author Yao Yuxiang

Maintainer Yao Yuxiang <y.x.yao@foxmail.com>

Description BioLogic, a user-friendly package designed to analyze the foundational properties of universal logical systems. It enables the examination of logical paradigms, computation of order parameters, transformation of system frameworks, and simulation of discrete dynamic systems.

License GPL-3 + MIT

Encoding UTF-8

LazyData true

Depends R (>= 3.5.0)

Imports Rcpp (>= 1.0.10),
ggplot2 (>= 3.4.0)

Suggests igraph

LinkingTo Rcpp

RoxygenNote 7.2.3

R topics documented:

`BoolBioNet_BoolFun` *Explore the features of Boolean functions in biological networks*

Description

This function computes and returns the category, sensitivity, effectiveness, and complexity of each Boolean function within a given Boolean network.

Usage

```
BoolBioNet_BoolFun(OneBioNet, Type = c("rough", "detail"))
```

Arguments

`OneBioNet`	A four-element formatted list that records information of a Boolean network.
`Type`	Which analysis method is applied, `rough` or `detail`?

Details

The function can identify various function types, including canalizing, threshold, clique, dominating, effective, monotone, signed, and spin-like functions. Definitions of these functions are available in the documentation of BoolFun_Type. Note that external factors are excluded from the analysis; only nodes with in-degrees between 2 and 15 (inclusive) are considered, while other cases return `NA`. When `Type` is set to `rough`, the function returns a `data.frame` containing summary statistics for each "in-degree" cluster. When `Type` is `detail`, the returned `data.frame` contains individual cases of the Boolean function.

Value

A `data.frame` summarizing the count of functions by type when `Type` is set to `rough` (or a `data.frame` listing the specific instances of each function within systems when `Type` is set to `detail`).

Examples

```
# Analysis of one real Boolean genetic network.
# BoolBioNet_BoolFun(BoolGRN_CellCollective$c_1778,"rough");
# Return a data.frame:
#                 Number Canalized Clique Monotone Signed Threshold
# InDegree_1           1        NA     NA       NA     NA        NA
# InDegree_2           3         3      3        1      3         3
# InDegree_3           3         3      2        0      3         3
#                              ... ...
BoolBioNet_BoolFun(BoolGRN_CellCollective$c_1778,"rough")

# BoolBioNet_BoolFun(BoolGRN_CellCollective$c_1778,"detail");
# Return a data.frame:
#            Input Canalized Clique Monotone Signed Threshold Sensitivity   ...
# ADD            5      TRUE  FALSE    FALSE   TRUE      TRUE   1.3125000   ...
# ATM            3      TRUE  FALSE    FALSE   TRUE      TRUE   1.2500000   ...
# ATR            4      TRUE  FALSE    FALSE   TRUE      TRUE   1.2500000   ...
#                                ... ...
BoolBioNet_BoolFun(BoolGRN_CellCollective$c_1778,"detail")
```

`BoolBioNet_Checker` *Check the correctness of the Boolean network configuration*

Description

The function evaluates the lengths of node names, input and output edges, and Boolean function configurations to assess the structural validity of the Boolean network.

Usage

```
BoolBioNet_Checker(OneBioNet)
```

Arguments

`OneBioNet`	A four-element formatted list that records information of a Boolean network.

Details

All information in the network is mutually consistent. For instance, the number of Boolean functions matches the number of nodes; if a node has k inputs, it implies that the node has a k-bit truth table (or Boolean function), etc. See BoolGRN_CellCollective.

Value

A logical value. If `TRUE`, it enables further downstream analysis. Abnormal results or invalid inputs will generate error messages to assist with troubleshooting.

Examples

```
# BoolBioNet_Checker(BoolGRN_CellCollective$c_1557);
# Return TRUE (Denote all information are appropriate)
BoolBioNet_Checker(BoolGRN_CellCollective$c_1557)
```

`BoolBioNet_CoreDyn` *Analyze dynamic core components of Boolean networks*

Description

This analysis only emphasizes the static feature of the network. The function recursively analyzes dynamic core components (`dyncore`) or relevant nodes (`relevant`) that contribute to systematic dynamic feature. These two indicators consider coupled feedforward loop or not, respectively.

Usage

```
BoolBioNet_CoreDyn(
  RealBioNet,
  AnalysisMod = c("dyncore", "relevant"),
  ExtSelfLoop = TRUE,
  Controller = NULL,
  ConVals = NULL,
  ExternalNode = NULL,
  NodeAttri = FALSE,
  ResidualNet = FALSE,
  Times = 100L
)
```

Arguments

`RealBioNet`	A four-element formatted list that records information of a Boolean network.
`AnalysisMod`	A string specifying either `dyncore` for dynamic core analysis or `relevant` for identifying all relevant nodes.
`ExtSelfLoop`	A logical value. Should inputs be converted into self-loops? (Default: `TRUE`)
`Controller`	An integer or character vector indicating the nodes to be controlled. (Default: `NULL`)
`ConVals`	A Boolean vector specifying the values of controlled of controlled nodes. (Default: `NULL`)
`ExternalNode`	A Boolean vector specifying the states of non-input nodes. (Default: `NULL`, indicating random assignment; see details for further information)
`NodeAttri`	A logical value. Should return node's attribute? (Default: `FALSE`, return `NA`)
`ResidualNet`	A logical value. Should return the residual network? (Default: `FALSE`, return `NA`)
`Times`	A positive integer representing the recursive number of analyses; users are advised not to modify this value.

Details

Some nodes in genetic networks lack inputs and are referred to as free nodes. If `ExtSelfLoop` is `TRUE`, self-loops are added to all free-nodes. When it is `FALSE`, the configuration of free nodes depends on `ExternalNode`. If `ExternalNode` is `NULL`, the states of the free nodes are fixed at randomly generated Boolean values (0 or 1). If provided (a Boolean vector), only the first `free node` elements are used; if the length of `ExternalNode` is less than `free node`, the function returns an error message and execution halts.

The Controller parameter specifies the manually controlled nodes, where either node indices or names are acceptable. Note that the function does not validate the correctness of the indices; users must ensure their accuracy. ConVals provides the corresponding values for the controlled genes. If ConVals is not specified (NULL), values are generated randomly. If provided (a Boolean vector), only the first |{Controller}| values are used; if the length of ConVals is less than that of Controller, the function returns an error message and execution halts. If a gene/node appears in both Controller and ExternalNode, the value specified in Controller takes precedence over that in ExternalNode.

ResidualNet retain all nodes of the original systems for comparative purposes (if ResidualNet is TRUE). Stable nodes, useless nodes, and their corresponding edges are removed in subsequent analyses. Only the remaining nodes, edges, and Boolean functions contribute to the terminal dynamic behaviors.

Value

A list containing four elements:

- [[1]] Overview: Sizes of [1] stable nodes, [2] invalid nodes, [3] valid nodes, [4] external components, and [5] terminal component.
- [[2]] Node_S01U: Node attribute 1 (IntegerVector or NA). Where "1", "2", "3" represent the node being stable at state 0, stable at state 1, and unstable, respectively.
- [[3]] Node_SUE: Node attribute 2 (IntegerVector or NA). Where "1", "0", "-1" represent that the node belongs to stable, useless, and engage types, respectively.
- [[4]] ResidualNetwork: The residual network is the analyzed network containing all nodes but with all invalid edges removed for comparative analysis.

Examples

```
# Analysis of dynamic core components of a Boolean genetic network.
# Treat external nodes as self-loop!
# BoolBioNet_CoreDyn(BoolGRN_CellCollective$c_11863, ExtSelfLoop=TRUE);
# Return [[1]] [1] 0, 28, 23, 0, 0
#        [[2]] ~ [[4]] NA
BoolBioNet_CoreDyn(BoolGRN_CellCollective$c_11863, ExtSelfLoop=TRUE)

# set.seed(1234);
# Treat free-node as fixed nodes!
# BoolBioNet_CoreDyn(BoolGRN_CellCollective$c_11863, ExtSelfLoop=FALSE);
# Return [[1]] [1] 51, 0, 0, 2, 0
#        [[2]] ~ [[4]] NA

set.seed(1234);
BoolBioNet_CoreDyn(BoolGRN_CellCollective$c_11863, ExtSelfLoop=FALSE)
```

BoolBioNet_FBLoops *Calculate the network's feedback loop*

Description

This function provides a brief analysis of feedback loops within the systems.

Usage

```
BoolBioNet_FBLoops(
  OneNet,
  AnalyType = c("loop", "length", "edgeattr", "loopsign"),
  Isolated = TRUE
)
```

Arguments

OneNet	A four-element formatted list that records information of a Boolean network.
AnalyType	A string indicating the feature of the feedback loop being analyzed. Only those "real" strongly connected components were used for analysis. (See BoolBioNet_StrConComp) • loop: Return the list of feedback loops. • length: Count the distribution of feedback loop lengths. • edgeattr: Analyze the fundamental properties of each edge in the loop, including the number of embedded loops, sign, edge activity, effectiveness. • loopsign: Return the sign property of each cycle. The sign attributes are categorized into three types: positive (+), negative (-), and hybrid (?). For the definitions of sign, edge activity, and effectiveness, refer to the following papers: [Aracena2008], [Kauffman2004], [Gates2021].
Isolated	A logical value. Should keep self-loops? (Default: TRUE)

Value

Depending on the value of AnalyType, the following types of return values may result:

- loop: a list, with the names of the elements indicating the SCC composition and the feedback loop.
- length: a matrix storing the distribution of feedback loop lengths.
- edgeattr: a datamframe characterizing the properties of each edge in the loop.
- loopsign: a matrix recording the sign attributes of each edge within each cycle.

Examples

```
# Analyze various properties of feedback loops in gene networks.
# Here is an example of the network [k_1753781] from Kadelka's paper.
# BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "loop")
# [[1]]scc1_loop1
# [1] "A"  "T2" "T1" "B"  "A"
# [[2]]scc1_loop2
# [1] "A"  "T2" "T1" "H"  "B"  "A"
# ... ...

# BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "length")
#      Loop_Length Count
# [1,]           1     6
# [2,]           2     8
# ... ...

# BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "edgeattr")
#    from to support SignType activity effectiveness
# 1     A T1       4        - 0.015625    0.14955357
```

```
# 2     A T2       8        + 0.031250    0.09505208
# ... ...

# BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "loopsign")
#      Positive Negative Hybrid
# [1,]        3        1      0
# [2,]        4        1      0
# ... ...

BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "loop")
BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "length")
BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "edgeattr")
BoolBioNet_FBLoops(BoolGRN_KadelkaSet$k_1753781, "loopsign")
```

`BoolBioNet_NodeAttr` *Display a node's attribute*

Description

This function returns the predecessors (inputs), successors (outputs), and regulatory patterns (Boolean function) of the specified nodes.

Usage

```
BoolBioNet_NodeAttr(RealBioNet, NodeName)
```

Arguments

`RealBioNet`	A four-element formatted list that records information of a Boolean network.
`NodeName`	Name of the node (gene).

Value

The information of the specified node.

Examples

```
# Print the information of the node "Akt" in the gene network [c_11863] from
# the CellCollective Set (BoolGRN_CellCollective).
# BoolBioNet_NodeAttr(BoolGRN_CellCollective$c_11863,"Akt");
# Return information:
# $TruthTable
#      PI3K [Akt]
# [1,]    0     0
# [2,]    1     1
# $Successor
# [1] "IKK"  "mTOR"
BoolBioNet_NodeAttr(BoolGRN_CellCollective$c_11863,"Akt")
```

BoolBioNet_SimpNet *Simplify networks*

Description

All isolated, redundant, or useless nodes are removed from the system, particularly following the analysis of core dynamic components, resulting in a simplified network.

Usage

```
BoolBioNet_SimpNet(OneNet)
```

Arguments

OneNet	A four-element formatted list that records information of a Boolean network.

Value

A four-element formatted list that records information of a Boolean network.

Examples

```
# Analysis of a real Boolean genetic network. The terms "Analyzed_Net" and
# "Simplified_Net" denote the network before and after simplification, respectively.
# Compare the two to understand their differences and the purpose of the
# simplification process.
set.seed(1234)
Analyzed_Net=BoolBioNet_CoreDyn(BoolGRN_CellCollective$c_2084, AnalysisMod="relevant",
  ExtSelfLoop=FALSE, ResidualNet=TRUE)[[4]]
Simplified_Net=BoolBioNet_SimpNet(Analyzed_Net)
# Original network.
BoolBioNet_Visualize(BoolGRN_CellCollective$c_2084, "print")
# Anlayzed network.
BoolBioNet_Visualize(Analyzed_Net, "print")
# Simplified network.
BoolBioNet_Visualize(Simplified_Net, "print")
```

BoolBioNet_StrConComp *Caluclate the network's strongly connected components*

Description

This function provides a brief analysis of the strongly connected components to facilitate comparison in core dynamic analysis.

Usage

```
BoolBioNet_StrConComp(OneNet)
```

Arguments

OneNet	A four-element formatted list that records information of a Boolean network.

Value

A list in which each element represents a strongly connected component. Where those labeled as _TRUE in the name are non-trivial SCCs, while _FALSE indicates terminal or relay nodes.

Examples

```
# Analyze the strongly connected components of a network.
# Here is an example of the network [c_2202] from the CellCollective Set
# BoolBioNet_StrConComp(BoolGRN_CellCollective$c_2202)
# Return a List
# [[1]]scc_1_FALSE   <-- a terminal node
# [1] "Exocytosis"
# [[2]]scc_2_FALSE   <-- a relay node
# [1] "Packaging_Proteins"
# [[3]]scc_3_TRUE <-- a "real" SCC that meets the requirement of signal feedback.
# [1] "Protein_Phosphatase_1" "DARPP32"              "Calcineurin"
# [4] "Calcium"               "Glutamate_Receptor"
# ... ...

BoolBioNet_StrConComp(BoolGRN_CellCollective$c_2202)
```

BoolBioNet_Visualize *Print or visualize networks*

Description

Convert the network into alternative standardized formats or visualizations, such as CharacterVector, data.frame, terminal output, or igraph objects (if available).

Usage

```
BoolBioNet_Visualize(
  RealBioNet,
  Outtype = c("print", "text", "dataframe", "igraph")
)
```

Arguments

RealBioNet	A four-element formatted list that records information of a Boolean network.
Outtype	A string indicating how the information will be structured or presented: • print: Display directly to the terminal. • text: A string that stores all information in the same format as the print scenario. • dataframe: A data frame recording source-target pairs. • igraph: An igraph object used to visualize the network (if the igraph package is available).

Value

Specifically formatted network data.

Examples

```
# Print the network [c_1969] from the CellCollective Set.
# BoolBioNet_Visualize(CellCollective$c_1969, "print");
# Print Out to terminal
#  "PI3K-> PTEN-> [Akt] ->Cdc42_Rac1"
#  "cFOS-> cJUN-> [AP1] ->uPAR"
#  "JNK-> p38-> [ATF2] ->CyclinD ->PTGS2"
#               ... ...
BoolBioNet_Visualize(BoolGRN_CellCollective$c_1969, "print")

# BoolBioNet_Visualize(BoolGRN_CellCollective$c_1969, "text");
# Return a text vector
#                                 Akt
# "PI3K-> PTEN-> [Akt] ->Cdc42_Rac1"
#                           AP1
# "cFOS-> cJUN-> [AP1] ->uPAR"
#          ... ...
BoolBioNet_Visualize(BoolGRN_CellCollective$c_1969, "text")

# BoolBioNet_Visualize(BoolGRN_CellCollective$c_1969, "dataframe");
# Return a data.frame
#         source        target
# 1         PI3K           Akt
# 2         PTEN           Akt
# 3         cFOS           AP1
#          ... ...
BoolBioNet_Visualize(BoolGRN_CellCollective$c_1969,"dataframe")
```

`BoolFun_Complexity` *Calculate the complexity of a Boolean function*

Description

The analysis depends on average of prime implicants in a Boolean function, based on Boolean Quine-McCluskey method.

Usage

```
BoolFun_Complexity(Bit_Vec)
```

Arguments

`Bit_Vec`	A Boolean vector (or 0/1-valued `IntegerVector`) represents the truth table of a Boolean function.

Value

A numeric value that denotes the complexity of a Boolean function.

Examples

```
# Calculate the complexity of a Boolean function \code{c(0,0,1,0,0,1,1,0)}.
# Based on Quine-McCluskey method:
# (*10)+(101) ==> 1
# (00*)+(*00)+(*11) ==> 0
# Thus, 2+3 =5 items as the complexity.
# BoolFun_Complexity(c(0,0,1,0,0,1,1,0))
# Return 5
BoolFun_Complexity(c(0,0,1,0,0,1,1,0))
```

`BoolFun_Effectiveness` *Calculate the input/edge effetiveness of a Boolean function*

Description

The function employs the Quine-McCluskey method to derive the prime implicants of input vectors for which $\{\vec{x}|f(\vec{x}) = 0\}$ and $\{\vec{x}|f(\vec{x}) = 1\}$, respectively.

Usage

```
BoolFun_Effectiveness(Bit_Vec, Detail = FALSE, Redundancy = FALSE)
```

Arguments

`Bit_Vec`	A Boolean vector (or 0/1-valued `IntegerVector`) represents the truth table of a Boolean function.
`Detail`	A logical value. Should return detail information of each edge? (Default: `FALSE`)
`Redundancy`	A logical value. Should return the corresponding redundant values of all edges rather than the effective ones? (Default: `FALSE`)

Details

The results can be obtained through either direct or indirect methods, which are equivalent in outcome. These computational processes are transparent to R users and do not impact usability. Advanced developers can utilize prototype functions to conduct various analyses based on specific requirements. Relevant concepts and definitions can be found in paper [Gates2021].

Value

A numeric value (or a NumericVector) denotes the effetive or redundant edge(s) of a Boolean function.

Examples

```
# Calculate the effetive input of a Boolean function c(0,0,0,0,0,0,1,1).
# Its can be transformed as: 0=f(*0*), 0=f(0**), and 1=f(11*).
# BoolFun_Effectiveness(c(0,0,0,0,0,0,1,1));
# Return 1.25000
BoolFun_Effectiveness(c(0,0,0,0,0,0,1,1))
```

```
# Calculate the effetive edges of a Boolean function c(0,0,0,1,1,1,1,1).
# Its can be transformed to 0=f(00*), 0=f(0*0), 1=f(1**) or f(*11)
# BoolFun_Effectiveness(c(0,0,0,1,1,1,1,1), Detail=TRUE);
# Return 0.3750 0.3750 0.8125
# Explain lowest/middle bit's 0.3750 and highest bit's 0.8125:
#  000 | 001 | 010 | 011 | 100 | 101 | 110 | 111  <== 2^3 vectors can be
#  00* | 00* |     |     | 1** | 1** | 1** | 1**      covered by which
#  0*0 |     | 0*0 | *11 |     |     |     | *11      prime implicants?
# (0.5 +   0 +   1 +   1 +   0 +   0 +   0 + 0.5 )/8 ==> 0.3750  # Lowest
# (0.5 +  1  +  0  +  1  +  0  +  0  +  0  + 0.5 )/8 ==> 0.3750  # Lowest
# ( 1  + 1   + 1   + 0   + 1   + 1   + 1   + 0.5 )/8 ==> 0.8125  # Highest bit

BoolFun_Effectiveness(c(0,0,0,1,1,1,1,1), Detail=TRUE)
```

`BoolFun_Expr2MapTable` *Convert Boolean expressions into mapping tables*

Description

The objective is to uncover the relationships between the mapping results of Boolean functions and their inputs, with implementation based on regular expressions.

Usage

```
BoolFun_Expr2MapTable(BoolExpChr, TableForm = FALSE)
```

Arguments

`BoolExpChr`	A string representing a valid Boolean expression.
`TableForm`	A logical value. Should return a table format? (`TRUE`: matrix; `FALSE`: Boolean vector)

Details

Note that the function does not verify the correctness of the Boolean expression.

Value

A Boolean vector or a mapping table.

Examples

```
# Convert \code{X = ((NOT A AND B)) OR C}.
# BoolFun_Expr2MapTable("X = ((NOT A AND B)) OR C");
# Return a BooleanVector c(FALSE, TRUE, TRUE, TRUE, FALSE, TRUE, FALSE, TRUE).
boolexp="X = ((NOT A AND B)) OR C"
BoolFun_Expr2MapTable(boolexp)

# Show-only \code{Y = (NOT A_a OR NOT B) AND (cc OR dd)}.
# BoolFun_Expr2MapTable("Y = (NOT A_a OR NOT B) AND (cc OR dd)", TRUE)
# Return a truth table:
#      A_a  B  cc  dd  Y
```

```
# [0]    0  0   0   0  0
# [1]    0  0   0   1  1
# [2]    0  0   1   0  1
#           ... ...
# Last column is c(0,1,1,1, 0,1,1,1, 0,1,1,1, 0,0,0,0)
boolexp="Y = (NOT A_a OR NOT B) AND (cc OR dd)"
BoolFun_Expr2MapTable(boolexp, TRUE)
```

`BoolFun_Generator` *Generate a specific type of Boolean function*

Description

Generate various specialized types of Boolean functions that exhibit greater order, efficiency, or redundancy compared to random functions. Detail definitions are provided in the papers mentioned in BoolFun_Type.

Usage

```
BoolFun_Generator(
  bf_type = c("R", "C", "P", "M", "D", "T"),
  VarNum = 3L,
  Bias = 0.5,
  vars = NULL
)
```

Arguments

`bf_type`	A character included in the following: • `C`: canalized • `D`: dominating • `M`: monotone • `P`: Post^2 (Clique) • `T`: Threshold • `R`: Random
`VarNum`	An integer between 1 and 15.
`Bias`	The fraction of "1" in the truth table of a Booelan function. Note that, except in random cases, the actual bias may differ from the set value due to the specific function type.
`vars`	Configure arguments for special type function: • an integrate: for `c('D','T')`; `'1'` or `'0'` for 1/0-type function, other means random set (random 1/0-type). • `c(int, int)`: for `'C'`, the first means layer of canalization; the second means 1/0-type or random (other number except for 1 and 0) canalized & canalizing values. • `c(int, int)`: for `'P'`, the first means 1/0-type or random as `c('D','T')` cases; the second determines whether repeated selection of a state is permitted (non-zero values indicate permission; zero indicates prohibition).

- `c(int, int)`: for `'M'`, the first means 1/0-type or random as `c('D','T')` cases; the second refers to the number of random seed (between 1 and `VarNum`).

Detail meaning see References of BoolFun_Type.

Value

An IntegerVector of length 2^{VarNum}.

Examples

```
# Generate a random function.
# set.seed(2024);
# BoolFun_Generator('R', 4L);
# Return c(0,1,0,0, 1,0,1,1, 0,1,0,0, 0,0,1,0)
set.seed(2024)
BoolFun_Generator('R', 4L)

# Generate a canalizing function.
# set.seed(202401);
# BoolFun_Generator('C', 3L);
# Return c(1,1,1,1, 1,0,0,0)
set.seed(202401)
BoolFun_Generator('C', 3L)
df_boolfun<-BoolFun_NestedCana(
  c(1,1,1,1, 1,0,0,0), PrintOut = TRUE)
# if {x_3=0} ==> f(x)=1
# else if {x_3=1, x_1=1} ==> f(x)=0
# else if {x_3=1, x_1=0, x_2=0} ==> f(x)=1
# else if {x_3=1, x_1=0, x_2=1} ==> f(x)=0

# Generate a threshold function.
# set.seed(202402);
# aboolfun=BoolFun_Generator('T', 5L);
# aboolfun is c(
#   1,1,0,1,0,1,0,0,
#   1,1,1,1,1,1,0,1,
#   0,1,0,0,0,0,0,0,
#   1,1,0,1,0,1,0,0)
# BoolFun_Type(aboolfun, 'T', TRUE);
# Return "a_0=1, a_1=-1, a_2=-1, a_3=1, a_4=-1, theta=1", TRUE
set.seed(202402)
aboolfun=BoolFun_Generator('T', 5L)
BoolFun_Type(aboolfun, 'T', TRUE)
```

BoolFun_NestedCana *Decode the structure of a nested canalized Boolean function*

Description

A nested canalized Boolean function typically exhibits an "if-else if..." regulatory structure. This function decodes and visually represents the pattern to illustrate its logical hierarchy intuitively.

Usage

```
BoolFun_NestedCana(Bit_Vec, PrintOut = FALSE)
```

Arguments

Bit_Vec	A Boolean vector (or 0/1-valued IntegerVector) represents the truth table of a Boolean function.
PrintOut	A logical value. Should output be displayed in the terminal? (Default: FALSE).

Value

A matrix that descirbes the nested structure of a canalized function.

Examples

```
# Show the canalized structure of a nested canalized function.
# set.seed(202501);
# aboolfun<-BoolFun_Generator("C", VarNum=7L, vars=c(4L,9L));
# df_boolfun<-BoolFun_NestedCana(aboolfun, PrintOut=TRUE);
# Return a matrix and display the nested relationships.
# if {x_5=1} ==> f(x)=0
# else if {x_5=0, x_6=1} ==> f(x)=0
# else if {x_5=0, x_6=0, x_7=1} ==> f(x)=0
# else if {x_5=0, x_6=0, x_7=0, x_3=1} ==> f(x)=1
# else if {x_5=0, x_6=0, x_7=0, x_3=0, x_4=1} ==> f(x)=0
# df_boolfun is:
#    x_1 x_2 x_3 x_4 x_5 x_6 x_7 Out
#  1  NA  NA  NA  NA   1  NA  NA   0
#  2  NA  NA  NA  NA   0   1  NA   0
#  3  NA  NA  NA  NA   0   0   1   0
#  4  NA  NA   1  NA   0   0   0   1
#  5  NA  NA   0   1   0   0   0   0

set.seed(202501)
aboolfun<-BoolFun_Generator("C", VarNum=7L, vars=c(4L,9L))
df_boolfun<-BoolFun_NestedCana(aboolfun, PrintOut=TRUE)
```

BoolFun_Polynomial *Convert a Boolean function into threshold-based or polynomial forms*

Description

Boolean functions are typically expressed as logical expressions involving variables and logical operators such as AND, OR, and NOT. When represented in arithmetic form, a Boolean function corresponds to a unique polynomial, including higher-order terms. It should be noted that the term *threshold-based* used here does not conform to the standard definition, which usually refers to models restricted to first-order terms.

Usage

```
BoolFun_Polynomial(Bit_Vec, SpinLikeForm = FALSE, PolyForm = FALSE)
```

Arguments

Bit_Vec	A Boolean vector (or 0/1-valued IntegerVector) represents the truth table of a Boolean function.
SpinLikeForm	A logical value. Should the Boolean function be converted into a spin-like form? (Default: FALSE)
PolyForm	A logical value. Should be converted according to a strict polynomial form.

Details

Boolean functions are typically represented in two standard forms: the binary form (0/1) and the spin-like form (+1/-1), which are linearly interconvertible. SpinLikeForm allows users to obtain representations in the spin-like form, while PolyForm provides an option to output the strict polynomial representation of a Boolean function. For example, for the expression "$f = a|b$", the polynomial form is "$f = a + b + ab$", and the threshold form is "$f = (a + b) > 0?$". See the provided examples for details. In the output, variables (x_1, x_2, ... , x_k, for a k-input function) represent the input bits ordered from the low bit to the high bit.

Value

A three-element formatted list:

- [[1]] Is Boolean satisfiability? (TRUE or FALSE)
- [[2]] The weight value of each variable and variable combination.
- [[3]] The highest order of terms contained in the function.

Examples

```
# Convert the a Boolean function into a polynomial expression:
# Case1: Boolean threshold
# BoolFun_Polynomial(c(0,1,1,0,1,0,0,0),F,F);
# Case2: Spin-like threshold
# BoolFun_Polynomial(c(0,1,1,0,1,0,0,0),T,F);
# Case3: Boolean strict polynomial form
# BoolFun_Polynomial(c(0,1,1,0,1,0,0,0),F,T);
# Case4: Spin-like strict polynomial form
# BoolFun_Polynomial(c(0,1,1,0,1,0,0,0),T,T);
# Return [[1]] sat:TRUE, [[2]] (see below), [[3]] HighOrder: 2 or 3
# Detail Weight of four types:
#         x_1    x_2    x_3  x_1x_2  x_1x_3  x_2x_3 x_1x_2x_3  thershold
# Case1     1      1      1      -2      -2      -2      NULL          0
# Case2    -2     -2     -2      -2      -2      -2      NULL          0
# Case3     1      1      1      -2      -2      -2         3          0
# Case4 -0.25  -0.25  -0.25   -0.25   -0.25   -0.25      0.75      -0.25

BoolFun_Polynomial(c(0,1,1,0,1,0,0,0), FALSE, FALSE) # Case1
BoolFun_Polynomial(c(0,1,1,0,1,0,0,0), TRUE,  FALSE) # Case2
BoolFun_Polynomial(c(0,1,1,0,1,0,0,0), FALSE, TRUE ) # Case3
BoolFun_Polynomial(c(0,1,1,0,1,0,0,0), TRUE,  TRUE ) # Case4
```

`BoolFun_QMForm` *Display the Quine-McCluskey form of a Boolean function*

Description

The function employs our native C++ standard Quine-McCluskey algorithm to obtain prime implicants. The function utilizes our native C++ implementation of the Quine-McCluskey algorithm to compute the prime implicants.

Usage

```
BoolFun_QMForm(Bit_Vec, VarsName = NULL)
```

Arguments

`Bit_Vec`	A Boolean vector (or 0/1-valued `IntegerVector`) represents the truth table of a Boolean function.
`VarsName`	A `CharacterVector` that represents each name of variable.

Details

The function prints the Quine-McCluskey form in the terminal, using uppercase and lowercase letters to indicate whether a variable is 1 or 0, respectively. Form low to high bits, variables are represented as `A/a`, `B/b`, `C/c`, ... If `VarsName` is provided, it must be of sufficient length to represent the bits from low to high; otherwise, it returns an error message and execution halts. Where the symbol "~" denotes the logical NOT.

Value

Show the Quine-McCluskey form of the given Boolean function.

Examples

```
# Calculate the Quine-McCluskey form of c(0,0,0,1,1,0,1,1).
# (a=0 and c=1) or (a=1 and b=1) ==> 1; (a->c, from low to high bit)
# BoolFun_QMForm(c(0,0,0,1,1,0,1,1));
# Return { 1 = aC + AB }
BoolFun_QMForm(c(0,0,0,1,1,0,1,1))

# Calculate the Quine-McCluskey form of c(1,1,0,1,1,1,0,0).
# (a=1 and c=0) or (b=0) ==> 1; (a->c, from low to high bit)
# BoolFun_QMForm(c(1,1,0,1,1,1,0,0), c("X1", "X2", "X3"));
# Return { 1= X1*~X3 + ~X2} (identical to { 1 = Ac + b })
BoolFun_QMForm(c(1,1,0,1,1,1,0,0), c("X1", "X2", "X3"))
```

BoolFun_Sensitivity *Calculate the sensitivity of a Boolean function*

Description

The analysis depends on enumerating input vectors and examining the mapping result of one-bit perturbations, such as $f(..., x_i = 0, ...)$ is identical to $f(..., x_i = 1, ...)$ or not. Detail concepts can see the paper [Shmulevich2004].

Usage

```
BoolFun_Sensitivity(Bit_Vec)
```

Arguments

Bit_Vec	A Boolean vector (or 0/1-valued IntegerVector) represents the truth table of a Boolean function.

Value

A numeric value that denotes the sensitivity of a Boolean function.

Examples

```
# Calculate the sensitivity of a Boolean function c(1,1,0,1,1,1,0,0).
# BoolFun_Sensitivity(c(1,1,0,1,1,1,0,0));
# Return 1.25000
BoolFun_Sensitivity(c(1,1,0,1,1,1,0,0))
```

BoolFun_Type *Determine whether a function belongs to a special type*

Description

This function determines whether the given function belongs to the specified category. For details, refer to the description of the parameter BF_Type.

Usage

```
BoolFun_Type(
  Bit_Vec,
  BF_Type = c("C", "P", "M", "S", "N", "D", "T", "E"),
  ShowRegulation = FALSE
)
```

Arguments

`Bit_Vec`	A Boolean vector (or 0/1-valued IntegerVector) represents the truth table of a Boolean function.
`BF_Type`	A character included in the following: • `C`: canalized [Kauffman2004] • `D`: dominating [Challet1998] • `M`: monotone [Korshunov2003] • `N`: Spin-like [Derrida1987; Fontclos,2018] • `P`: Post^2 (Clique) [Pogosyan1997] • `S`: Sign [Aracena2008] • `T`: Threshold [Li2004]
`ShowRegulation`	A logical value. Should the detailed information for T/N-type be displayed? (Default: `FALSE`)

Value

A logical value. Whether the given function belongs to the specified type.

Examples

```
# Is it a Canalized function? [Canalizing/Canalized value is 1/0]
# BoolFun_Type(c(runif(8)>0.5,rep(FALSE,each=8)),'C');
# Return a TRUE.
BoolFun_Type(c(runif(8)>0.5,rep(FALSE,each=8)),'C')

# Is it a Post function? [y=x1*x2+x2*x3+x3*x1] ~ c(0,0,0,1,0,1,1,0)
# BoolFun_Type(c(0,0,0,1,0,1,1,0),'P');
# Return a TRUE.
BoolFun_Type(c(0,0,0,1,0,1,1,0),'P')

# Is it a Threshold function?
# The given function originates from "(-x1+x2-x3+1>0)?TRUE:FALSE".
# BoolFun_Type(c(1,0,1,1,0,0,1,0),'T');
# Return a TRUE.
BoolFun_Type(c(1,0,1,1,0,0,1,0),'T')
```

`BoolGRN_CellCollective` *70 genetic networks selected from Cell Collective database*

Description

A list containing networks from the database, with each element representing an independent network. The number is consistent with that provided on the website. For example, `c_11863` refers to the biological network No.11863.

Usage

```
data(BoolGRN_CellCollective)
```

Format

A list of genetic networks.

Details

Each network is organized as a four-element formatted list:

- `AllMember`: a `CharacterVector` that record all names of gene.
- `InEdge`: a `list` of `IntegerVector`; each vector records the indices of input control nodes for each node, with the first element corresponding to the highest bit in the truth table. No input is denoted as `NA`.
- `OutEdge`: a `list` of `IntegerVector`; each vector records downstream nodes. No output is denoted as `NA`.
- `BoolFun`: a `list` of `IntegerVector`; each vector represents the Boolean function with length of (`BoolFun[[i]]` is identical to `2^length(InEdge[[i]])`). If the coressponing `InEdge[[i]]` is `NA`, it remains `NA`.

Please note that node indices start from 0 rather than 1. Other detail information about the database, please see Cell Collective Website.

`BoolGRN_KadelkaSet` *91 genetic networks in Kadelka's paper*

Description

A list containing networks from the paper, with each element representing an independent network. The number is consistent with the PMID of original reference. For example, `k_11082279` represents the paper "PMID:11082279".

Usage

```
data(BoolGRN_KadelkaSet)
```

Format

An object of class `list` of length 91.

Details

Each element is a four-element formatted list that records information of a Boolean network (See BoolGRN_CellCollective). Detail information about dataset, please see original paper Kadelka2024

BoolGRN_ThresholdModel *8 threshold-based genetic networks*

Description

A list containing networks from the papers, with each element representing an independent network. Their names briefly describe the features of the networks.

Usage

```
data(BoolGRN_ThresholdModel)
```

Format

An object of class `list` of length 8.

Details

The source of the network is provided in the references:

- `ArabidopsisFlower`: The network determines during Arabidopsis thaliana flower development Espinosa-Soto2004
- `emt26network`: 36-node epithelial-to-mesenchymal transition (EMT) network Steinway2015
- `emt72network`: 72-node EMT network Jia2019
- `gastricnetwork`: Endogenous molecular network of gastric cancer Li2015
- `sclcnetwork`: Transcription factor network in small cell lung cancer Udyavar2016
- `stemcellnetwork`: Network of induced pluripotent stem cells Chang2011
- `yeastbudding`: Budding yeast cell cycle network Li2004
- `yeastfission`: Fission yeast cell cycle network Davidich2008

Each element is a four-element formatted list that records information of a Boolean network (See BoolGRN_CellCollective). Other detail information about genetic networks, please see original papers.

c_BF_Complexity *Calculate the complexity of a Boolean function*

Description

C++ prototype function of BoolFun_Complexity.

Usage

```
c_BF_Complexity(boolfuns, k)
```

Arguments

`boolfuns`	[`IntegerVector &`] the Boolean function.
`k`	[`int`] the number of input variables of the function.

c_BF_Effective *Calculate the effective edges of a Boolean function*

Description

C++ prototype function of BoolFun_Effectiveness.

Usage

```
c_BF_Effective(boolfuns, k)
```

Arguments

boolfuns	[IntegerVector &] the Boolean function.
k	[int] the number of input variables of the function.

c_BF_EffectiveEdges *Calculate the effective edges of a Boolean function*

Description

C++ prototype function of BoolFun_Effectiveness.

Usage

```
c_BF_EffectiveEdges(boolfuns, k)
```

Arguments

boolfuns	[IntegerVector &] the Boolean function.
k	[int] the number of input variables of the function.

c_BF_Generator *Generate a specific type of Boolean function*

Description

C++ prototype function of BoolFun_Generator.

Usage

```
c_BF_Generator(Type, k, bias, Vars)
```

Arguments

Type	[char] function type.
k	[k] the number of input variables of the function.
bias	[double] function bias.
Vars	[IntegerVector] some configuration parameters.

c_BF_isPointed *Determine whether a function belongs to a special type*

Description

C++ prototype function of BoolFun_Type.

Usage

```
c_BF_isPointed(boolfuns, k, Type, Showit)
```

Arguments

boolfuns	[LogicalVector] the Boolean function logical vector.
k	[int] the number of input variables of the function.
Type	[char] function type.
Showit	[bool] whether show the function?

c_BF_QuineMcCluskey *Display the Quine-McCluskey form of a Boolean function*

Description

C++ prototype function of BoolFun_QMForm.

Usage

```
c_BF_QuineMcCluskey(boolfuns, k, VarsName)
```

Arguments

boolfuns	[IntegerVector &] the Boolean function.
k	[int] the number of input variables of the function.
VarsName	[CharacterVector] variable names.

c_BF_Sensitivity *Calculate the sensitivity of a Boolean function*

Description

C++ prototype function of BoolFun_Sensitivity.

Usage

```
c_BF_Sensitivity(boolfuns, k)
```

Arguments

boolfuns	[LogicalVector &] the Boolean function logical vector.
k	[int] the number of input variables of the function.

c_BoolFun2Polynomial *Convert a Boolean function into threshold-based or polynomial forms*

Description

C++ prototype function of BoolFun_Polynomial.

Usage

```
c_BoolFun2Polynomial(VariableMat, MapTab, LogiSpin)
```

Arguments

VariableMat	[IntegerMatrix &] a matrix of variable combination
MapTab	[IntegerMatrix &] the truth table of Boolean function.
LogiSpin	[int] Should show the detail polynomial form?

c_B_NestedCanalized *Decode the structure of a nested canalized Boolean function*

Description

C++ prototype function of BoolFun_NestedCana.

Usage

```
c_B_NestedCanalized(boolfuns, k)
```

Arguments

boolfuns	[IntegerVector &] the Boolean function logical vector.
k	[int] the number of input variables of the function.

c_CoreDynamicNode *Analyze core dynamic components of Boolean networks (1)*

Description

C++ prototype function of BoolBioNet_CoreDyn (coredyn part).

Usage

```
c_CoreDynamicNode(
  aRealNet,
  PointedGene,
  PointValues,
  InD,
  OtD,
  NodeDetailInfor,
  ReturnResidualNetwork,
  Times
)
```

Arguments

aRealNet	[List] a real/silico genetic network (See BoolGRN_CellCollective).
PointedGene	[IntegerVector] which nodes should be fixed.
PointValues	[IntegerVector] pointed values of free/controlled nodes.
InD	[IntegerVector] the vector of in-degrees.
OtD	[IntegerVector] the vector of out-degrees.
NodeDetailInfor	[int] should return detail information of node (>0, yes).
ReturnResidualNetwork	[int] should return Residual Network (>0, yes).
Times	[int] recursive number of analysis (>0).

---

c_Derrida_Simualtion *Analyze damage spread within system*

Description

C++ prototype function of DNS_DamageSpread

Usage

```
c_Derrida_Simualtion(
  sys_size,
  ll_system,
  sim_step,
  obf_type,
  bias_rf,
  obf_ratio,
  init_dis,
  init_1_ratio,
  net_type,
  net_f_para,
  obf_i_para1,
  obf_i_para2,
  RuleType
)
```

Arguments

sys_size	[int] system size
ll_system	[int] level number of discrete system
sim_step	[int] simulating steps
obf_type	[char] ordered Boolean function type
bias_rf	[NumericVector] bias in random function
obf_ratio	[double] proportion of ordered Boolean function
init_dis	[double] initial normalized Hamming distance
init_1_ratio	[NumericVector] "1" proportion of initial state
net_type	[char] system topological type
net_f_para	[double] topological configured parameters
obf_i_para1	[int] configuration parameter for OBF (1)
obf_i_para2	[int] configuration parameter for OBF (2)
RuleType	[int] update strategy

---

c_FrameTruthTable *Enumerate all possible input vectors*

Description

C++ prototype function for enumerate all possible input vectors under the given length.

Usage

```
c_FrameTruthTable(VarNum)
```

Arguments

VarNum	[int] the number of input variables of the function

---

c_MulF_Complexity *Calculate the complexity of a multi-valued function*

Description

C++ prototype function of MulVFun_Complexity

Usage

```
c_MulF_Complexity(avec, k, L)
```

Arguments

avec	[IntegerVector] the function.
k	[int] the number of input variables.
L	[int] the level of function.

Value

`double` the complexity of given function.

`c_MulF_Effective` *Calculate the effetive edges loading of a multi-valued function*

Description

C++ prototype function (1) of MulVFun_Effectiveness

Usage

```
c_MulF_Effective(avec, k, L)
```

Arguments

`avec`	[`IntegerVector`] the function.
`k`	[`int`] the number of input variables.
`L`	[`int`] the level of function.

Value

`double` the global effetive or redundant edges of given function.

`c_MulF_EffectiveEdges` *Calculate the effetive edges loading of a multi-valued function*

Description

C++ prototype function (2) of MulVFun_Effectiveness

Usage

```
c_MulF_EffectiveEdges(avec, k, L)
```

Arguments

`avec`	[`IntegerVector`] the function.
`k`	[`int`] the number of input variables.
`L`	[`int`] the level of function.

Value

`NumericVector` the effetive or redundant value of each edge.

c_MulF_QuineMcCluskey *Show the Quine-McCluskey form of a multi-valued function*

Description

C++ prototype function of MulVFun_QMForm

Usage

```
c_MulF_QuineMcCluskey(avec, k, L)
```

Arguments

avec	[IntegerVector &] that represents a mapping table of multi-valued function.
k	[int] that denotes the number of input variables.
L	[int] that represents the level of multi-valued system.

c_MulV2Bool_Bool2MulV *Boolean to multi-valued and multi-valued to Boolean transformation*

Description

C++ prototype function of MulV2Bool_Bool2MulV

Usage

```
c_MulV2Bool_Bool2MulV(OriMapTab, k, L, Thresholds, b2m)
```

Arguments

OriMapTab,	[IntegerVector] original mapping table.
k,	[int] the number of input variables.
L,	[int] the level of multi-valued system.
Thresholds,	[IntegerVector] thresholds for a multi-valued system.
b2m,	[int] Is from Boolean to Multi-valued system? (>0, yes)

c_MulVFun2Polynomial *Convert a multi-valued function as polynomial like forms*

Description

C++ prototype function of MulVFun_Polynomial

Usage

```
c_MulVFun2Polynomial(VariableMat, MapTab, k, L)
```

Arguments

VariableMat	[IntegerMatrix &] a matrix that record input vectors.
MapTab	[IntegerVector &] the function.
k	[int] the number of input variables of original function.
L	[int] the level of original function.

c_MulVF_Generator *Generate a specific type of multi-valued function*

Description

C++ prototype function of MulVFun_Generator

Usage

```
c_MulVF_Generator(
  FunType,
  k,
  L,
  CanaDeep,
  CanaVar,
  CanaVarNum,
  CanaInfo1,
  CanaInfo2,
  bias,
  Cana_Free
)
```

Arguments

FunType	[char] function type.
k	[int] the number of input variables.
L	[int] the level of function.
CanaDeep	[int] the layer of canalization.
CanaVar	[IntegerVector &] canalized variable indexes.

CanaVarNum	[IntegerVector &] the canalizing number of each variable.
CanaInfo1	[List &] canalizing information.
CanaInfo2	[List &] canalized information.
bias	[NumericVector &] function bias.
Cana_Free	[bool] is a parameter-free canalized function?

---

c_MulVF_Sensitivity *Calculate the sensitivity of a multi-valued function*

Description

C++ prototype function of MulVFun_Sensitivity

Usage

```
c_MulVF_Sensitivity(amulfun, k, L, Lens)
```

Arguments

amulfun	[IntegerVector &] the function.
k	[int] the number of input variables.
L	[int] the level of function.
Lens	[int] the length of mapping table (L^k).

---

c_M_Domainted *Decode the information of a multi-valued domaineted function*

Description

C++ prototype function of MulVFun_is_Domainted

Usage

```
c_M_Domainted(amulfun, k, L, Lens)
```

Arguments

amulfun	[IntegerVector &] the function.
k	[int] the number of input variables of original function.
L	[int] the level of original function.
Lens	[int] the length of mapping table (L^k).

c_M_NestedCanalized *Analyze the structure of a multi-valued nested canalized function*

Description

C++ prototype function of MulVFun_is_NestedCana

Usage

```
c_M_NestedCanalized(amulfun, k, L, Lens)
```

Arguments

amulfun	[IntegerVector &] the function.
k	[int] the number of input variables.
L	[int] the level of function.
Lens	[int] the length of mapping table (L^k).

c_M_Signed *Decode the information of a multi-valued signed function*

Description

C++ prototype function of MulVFun_is_Signed

Usage

```
c_M_Signed(amulfun, k, L, Lens)
```

Arguments

amulfun	[IntegerVector &] the function.
k	[int] the number of input variables of original function.
L	[int] the level of original function.
Lens	[int] the length of mapping table (L^k).

c_M_Threshold *Decode the information of a multi-valued linear threshold function*

Description

C++ prototype function of MulVFun_is_Threshold

Usage

```
c_M_Threshold(amulfun, k, L, Lens)
```

Arguments

amulfun	[IntegerVector &] the function.
k	[int] the number of input variables.
L	[int] the level of function.
Lens	[int] the length of mapping table (L^k).

c_Percolation_Simualtion *Analyze percolation within system*

Description

C++ prototype function of DNS_Percolation

Usage

```
c_Percolation_Simualtion(
  sys_size,
  ll_system,
  sim_step,
  lat_type,
  obr_window,
  obf_type,
  bias_rf,
  obf_ratio,
  init_bias,
  net_type,
  net_f_para,
  obf_i_para1,
  obf_i_para2,
  OutPut,
  RuleType
)
```

Arguments

sys_size	[int] system size
ll_system	[int] level number of discrete system
sim_step	[int] simulating steps
lat_type	[int] lattice type
obr_window	[int] observe window
obf_type	[char] ordered Boolean function type
bias_rf	[NumericVector] biases in random function
obf_ratio	[double] proportion of ordered Boolean function
init_bias	[NumericVector] "1" proportion of initial state
net_type	[char] system topological type
net_f_para	[char] topological configured parameters
obf_i_para1	[int] configuration parameter for OBF (1)
obf_i_para2	[int] configuration parameter for OBF (2)
OutPut	[bool] whehther output all states?
RuleType	[int] update strategy

---

c_ScalingLaw_RealNet *Analyze core dynamic components of Boolean networks (2)*

Description

C++ prototype function of BoolBioNet_CoreDyn (scaling part).

Usage

```
c_ScalingLaw_RealNet(
  aRealNet,
  PointedGene,
  PointValues,
  InD,
  OtD,
  NodeDetailInfor,
  ReturnResidualNetwork
)
```

Arguments

aRealNet	[List] a real/silico genetic network (See BoolGRN_CellCollective).
PointedGene	[IntegerVector] which nodes should be fixed.
PointValues	[IntegerVector] pointed values of free/controlled nodes.
InD	[IntegerVector] the vector of in-degrees.
OtD	[IntegerVector] the vector of out-degrees.
NodeDetailInfor	[int] should return detail information of node (>0, yes).
ReturnResidualNetwork	[int] recursive number of analysis (>0).

`c_ScalingLaw_Simualtion` *Analyze potential engaged nodes within Boolean/multi-valued system*

Description

C++ prototype function of DNS_Engaged

Usage

```
c_ScalingLaw_Simualtion(
  sys_size,
  ll_system,
  obf_type,
  bias_rf,
  obf_ratio,
  net_type,
  net_f_para,
  obf_i_para1,
  obf_i_para2,
  PointedNode,
  PointValues,
  NodeDetailInfor,
  ReturnResidualNetwork
)
```

Arguments

`sys_size`	[`int`] system size
`ll_system`	[`int`] level number of discrete system
`obf_type`	[`char`] ordered Boolean function type
`bias_rf`	[`NumericVector`] biases in random function
`obf_ratio`	[`double`] proportion of ordered Boolean function
`net_type`	[`char`] system topological type
`net_f_para`	[`char`] topological configured parameters
`obf_i_para1`	[`int`] configuration parameter for OBF (1)
`obf_i_para2`	[`int`] configuration parameter for OBF (2)
`PointedNode`	[`IntegerVector`] which nodes should be fixed
`PointValues`	[`IntegerVector`] pointed values of free/controlled nodes
`NodeDetailInfor`	[`bool`] should return detail informations of node
`ReturnResidualNetwork`	[`bool`] should return residual networks?

c_StrongConnectComponent *Caluclate the network's strongly connected components*

Description

C++ prototype function of BoolBioNet_StrConComp

Usage

```
c_StrongConnectComponent(aRealNet, InD, OtD)
```

Arguments

aRealNet	[List] a real/silico genetic network (See BoolGRN_CellCollective).
InD	[IntegerVector] the vector of in-degrees.
OtD	[IntegerVector] the vector of out-degrees.

DNS_DamageSpread *Analyze damage spread within system*

Description

The Derrida curve is employed to quantify the effect of damage. The procedure involves: generating two system states with a specified Hamming distance; allowing both states to evolve independently under identical update rules; and measuring their final Hamming distance after a fixed number of steps. Further details on this concept can concepts can be found in [Kauffman2004].

Usage

```
DNS_DamageSpread(
  Size = 1000L,
  SimStep = 1000L,
  Init_Dist = 0.1,
  Init_1_Ratio = c(0.5, 0.5),
  OBF_Type = "R",
  OBF_iPara1 = 1L,
  OBF_iPara2 = 1L,
  OBF_Ratio = 0.1,
  RBF_Bias = c(0.5, 0.5),
  Net_Type = "K",
  Net_fPara = 4,
  NumSys = 2L,
  UpdateRule = 1L
)
```

Arguments

Size	an integer, size of system.
SimStep	an integer, steps of simulating system.
Init_Dist	a numeric value, initial normalized Hamming distance of two vector.
Init_1_Ratio	a NumvericVector, proportion of each value (its length is NumSys).
OBF_Type	a character, ordered Boolean function type (OBF).
OBF_iPara1	an integer, configuration parameter for OBF.
OBF_iPara2	an integer, configuration parameter for OBF (Not necessary for some types of OBF).
OBF_Ratio	a numeric value, proportion of ordered Boolean function within system.
RBF_Bias	a NumvericVector, biases of random function (each is (0,1), sum=1).
Net_Type	a character, system topological type.
Net_fPara	a numeric value, topological configured parameters.
NumSys	an integer, number of discrete value.
UpdateRule	an integer, update rule: 1=synchronous, 2=asynchronous, 3=quick-asynchronous.

Details

Ensure that all parameters are properly set. Size, SimStep, Init_Dist, Init_1_Ratio are dynamic parameters for simualtion. To avoid transient states, it recommends SimStep >= Size. Init_Dist has little effect on the final distance, while Init_1_Ratio can cause difference in some cases (when OBF is D-type). OBF_Type, OBF_iPara1, OBF_iPara2, RBF_Bias, OBF_Ratio, configure functions. See their roles in BoolFun_Generator. Net_Type and Net_fPara determine the topological structure:

- K: Kauffman model, Net_fPara is in-degree.
- E: Erdos-Renyi graph, Net_fPara is average degree.
- R: Regular random graph, Net_fPara is connecting number.
- L: Lattic square, Net_fPara is type (4=Square; 3=Triangle; 6=Hexagon).
- N: Null model (Not enabled here).

Value

A numeric value that representing the normalized Hamming distance.

Examples

```
# set.seed(20250101L);
# DNS_DamageSpread(Size=1000L, SimStep=1000L, OBF_Type='C',
#   OBF_iPara1=1, OBF_iPara2=-1, OBF_Ratio=0.5);
# Return 0.335
set.seed(20250101L)
DNS_DamageSpread(Size=1000L, SimStep=1000L, OBF_Type='C',
  OBF_iPara1=1, OBF_iPara2=-1, OBF_Ratio=0.5)
```

DNS_DamageSpread_Load *Analyze damage spread within system (Loaded systems)*

Description

Same as DNS_DamageSpread, but the network is loaded.

Usage

```
DNS_DamageSpread_Load(
  RealBioNet,
  ExtSelfLoop = TRUE,
  Controller = NULL,
  ConVals = NULL,
  ExternalNode = NULL,
  Times = 1000L,
  Init_Dist = 0.1,
  Init_1_Ratio = c(0.5, 0.5),
  NumSys = 2L,
  UpdateRule = 1L
)
```

Arguments

RealBioNet	A four-element formatted list that records information of a Boolean network.
ExtSelfLoop	A logical value. Should inputs be converted into self-loops? (Default: TRUE)
Controller	An integer or character vector indicating the nodes to be controlled. (Default: NULL)
ConVals	A Boolean vector specifying the values of controlled of controlled nodes. (Default: NULL)
ExternalNode	A Boolean vector specifying the states of non-input nodes. (Default: NULL, indicating random assignment)
Times	an integer, steps of simulating system.
Init_Dist	a numeric value, initial normalized Hamming distance of two vector.
Init_1_Ratio	a NumvericVector, proportion of each value (its length is NumSys).
NumSys	an integer, number of discrete value.
UpdateRule	an integer, update rule: 1=synchronous, 2=asynchronous, 3=quick-asynchronous.

Details

See DNS_DamageSpread.

Value

A numeric value that representing the normalized Hamming distance.

Examples

```
# set.seed(20250101L);
# DNS_DamageSpread(Size=1000L, SimStep=1000L, OBF_Type='C',
#   OBF_iPara1=1, OBF_iPara2=-1, OBF_Ratio=0.5);
# Return 0.335
# set.seed(20250101L)
# DNS_DamageSpread(Size=1000L, SimStep=1000L)
```

`DNS_Engaged` *Analyze potential engaged nodes within Boolean/multi-valued system*

Description

The function recursively analyzes nodes that contribute to the final systematic dynamic feature. Nodes are not considered as "engaged", including terminal nodes (those with out-degree zero) and useless nodes (those whose outgoing links are all useless). A node is deemed *Useless* if it does not influence the computation of any downstream node. For example, given $\{A, B, C\} \rightarrow X$, where $f(X) = A * B$, thus node C is useless because it does not affect the value of X (if and only if C does not have other downstream nodes). Detail concepts can be found in the paper [Socolar2003].

Usage

```
DNS_Engaged(
  Size = 1000L,
  OBF_Type = "R",
  OBF_iPara1 = 1L,
  OBF_iPara2 = 1L,
  OBF_Ratio = 0.1,
  RBF_Bias = c(0.5, 0.5),
  Net_Type = "K",
  Net_fPara = 4,
  Controller = NULL,
  ConVals = NULL,
  NodeAttri = FALSE,
  ResidualNet = FALSE,
  NumSys = 2L
)
```

Arguments

`Size`	an integer, size of system.
`OBF_Type`	a character, ordered Boolean function type (OBF).
`OBF_iPara1`	an integer, configuration parameter for OBF.
`OBF_iPara2`	an integer, configuration parameter for OBF (Not necessary for some types of OBF).
`OBF_Ratio`	a numeric value, proportion of ordered Boolean function within system.
`RBF_Bias`	a NumvericVector, biases of random function (each is (0,1), sum=1)
`Net_Type`	a character, system topological type.

Net_fPara	a numeric value, topological configured parameters.
Controller	an IntegerVector, denote which nodes should be manually controlled. (Default: NULL)
ConVals	a BooleanVector, controlled nodes' values. (Default: NULL)
NodeAttri	A logical value, should return node's attribute? (Default: FALSE)
ResidualNet	A logical value, should return residual network? (Default: FALSE)
NumSys	an integer, number of discrete value

Details

Ensure that all parameters are properly set. Size is dynamic parameters for simualtion. OBF_Type, OBF_iPara1, OBF_iPara2, RBF_Bias, OBF_Ratio, configure functions. See their roles in BoolFun_Generator. Net_Type and Net_fPara determine the topological structure:

- K: Kauffman model, Net_fPara is in-degree.
- E: Erdos-Renyi graph, Net_fPara is average degree.
- R: Regular random graph, Net_fPara is connecting number.
- L: Lattic sqaure, Net_fPara is type (4=Square; 3=Triangle; 6=Hexagon).
- N: Null model (Not enabled here).

The Controller parameter specifies the manually controlled nodes, where either node indices or names are acceptable. Note that the function does not validate the correctness of the indices; users must ensure their accuracy. ConVals provides the corresponding values for the controlled genes. If ConVals is not specified (NULL), values are generated randomly. If provided (a Boolean vector), only the first |{Controller}| values are used; if the length of ConVals is less than that of Controller, the function returns an error message and execution halts.

Residual networks retain all nodes from the original systems for comparative purposes (if ResidualNet is TRUE). Stable nodes, useless nodes, and their corresponding edges are removed in subsequent analyses. Only the remaining nodes, edges, and Boolean functions contribute to the terminal dynamic behaviors.

Value

A five-element list:

- [[1]]: overall information (See example).
- [[2]] IntegerVector[Size]: the detail information regarding the possible coexistence states of each node is represented as an int32 number, where each bit indicates the presence of a corresponding state in the node.
- [[3]] IntegerVector[Size]: record all nodes are stable(1), useless(0), engaged(-1).
- [[4]] IntegerVector[Size]: the number of coexistence states of each node, such as stable(1), two-possible(2), three-possible(3) ... (>2 means multi-valued systems)
- [[5]]: the residual network structure.

Examples

```
# Test a (k=2, p=0.5) Kauffman model. It is critical when 2p(1-p)K=1.
# set.seed(20250101L);
# DNS_Engaged(10000L, RBF_Bias=0.5, Net_Type='K', Net_fPara=2.00);
# Return $[[Overview]] (other are NA)
```

```
#  Stable    Useless    Engaged   External Controlled
#    7641       1713        646          0          0
set.seed(20250101L)
DNS_Engaged(10000L, RBF_Bias=0.5, Net_Type='K', Net_fPara=2.00)

# Test a (k=3, q=0.788) Kauffman model. It is critical when 2p(1-p)K~1.
# set.seed(20250102L);
# DNS_Engaged(10000L, RBF_Bias=0.788, Net_Type='K', Net_fPara=3.00);
# Return $[[Overview]] (other are NA)
#  Stable    Useless    Engaged   External Controlled
#    7991       1171        838          0          0
set.seed(20250102L)
DNS_Engaged(10000L, RBF_Bias=0.788, Net_Type='K', Net_fPara=3.00)

# Test a (k=3, q=0.5) Kauffman model. It is chaotic, when 2p(1-p)K>1.
# set.seed(20250103L);
# DNS_Engaged(10000L, RBF_Bias=0.5, Net_Type='K', Net_fPara=3.00);
# Return $[[Overview]] (other are NA)
#   Stable    Useless    Engaged   External Controlled
#      118        732       9150          0          0
set.seed(20250103L)
DNS_Engaged(10000L, RBF_Bias=0.5, Net_Type='K', Net_fPara=3.00)
```

`DNS_Percolation` *Analyze percolation within system*

Description

Here employs maximal stable components (MSC) of Boolean networks embedded in square lattice as measure of percolations. A random state of a given Boolean network finally fall in an attractor with stable or oscillatory pattern. Here observe the MSC can form a continuous path as the occurrence of percolation. Please note that this definition sources from [Shmulevich2003]. There are also other definitions of so-called "percolation".

Usage

```
DNS_Percolation(
  Size = 1000L,
  SimStep = 1000L,
  ObsWin = 1000L,
  OutPutState = FALSE,
  OBF_Type = "R",
  OBF_iPara1 = 1L,
  OBF_iPara2 = 1L,
  OBF_Ratio = 0.1,
  RBF_Bias = c(0.5, 0.5),
  Net_fPara = 4,
  Init_1_Ratio = c(0.5, 0.5),
  NumSys = 2L,
  LatType = 4L,
  UpdateRule = 1L
)
```

Arguments

Size	an integer, size of system.
SimStep	an integer, steps of simulating system.
ObsWin	an integer, the size of observing window to ensure the stable cluster.
OutPutState	a logical value, should output all states? (Default: FALSE)
OBF_Type	a character, ordered Boolean type (OBF)
OBF_iPara1	an integer, configuration parameter for OBF.
OBF_iPara2	an integer, configuration parameter for OBF (Not necessary for some types of OBF).
OBF_Ratio	a numeric value, proportion of ordered Boolean function within system.
RBF_Bias	a NumvericVector, biases of random function (each is (0,1), sum=1).
Net_fPara	a numeric value, topological configured parameters.
Init_1_Ratio	a NumvericVector, proportion of each value (its length is NumSys).
NumSys	an integer, number of discrete value.
LatType	an integer, 4:square, 6: hexangular, 3:triangular
UpdateRule	an integer, update rule: 1=synchronous, 2=asynchronous, 3=quick-asynchronous.

Details

Ensure that all parameters are properly set. Some parameters are fixed due to specific tasks. Size, SimStep, ObsWin are dynamic parameters for simualtion. To avoid transient states, recommend SimStep >= Size. OBF_Type, OBF_iPara1, OBF_iPara2, RBF_Bias, OBF_Ratio, configure functions. See their roles in BoolFun_Generator. Net_fPara control lattice type: (4, Square; 3, triangle; 6, hexagon); here Size is an even squared number.

Value

A three-element list:

- [[1]] NumericVector[2]: [1] max stable cluster fraction (non-zero means percolation happens); [2] stable node fraction.
- [[2]] IntegerVector[Size]: Stable or unstable states of all nodes. Value "1", "2", "3" represent the node being stable at state 0, stable at state 1, and unstable, respectively. (if OutPutState is TRUE).
- [[3]] IntegerVector[Size]: record if each stable nodes belong the maximal stable components (MSC).

Examples

```
# Test percolation random and canalized functions (70%)
# set.seed(20250101L);
# DNS_Percolation(2500L, 2500L, 2000L, OBF_Type='C',
#   OBF_iPara1=2, OBF_iPara2=-1, OBF_Ratio=0.7);
# Return [[1]] [1] 0.6456 0.6628 (Percolation)
set.seed(20250101L)
DNS_Percolation(2500L, 2500L, 2000L, OBF_Type='C',
  OBF_iPara1=2, OBF_iPara2=-1, OBF_Ratio=0.7)

# Test percolation random and canalized functions (30%)
```

```
# set.seed(20250102L);
# DNS_Percolation(2500L, 2500L, 2000L, OBF_Type='C',
#  OBF_iPara1=2, OBF_iPara2=-1, OBF_Ratio=0.3);
# Return [[1]] [1] 0.0000 0.3448 (No percolation)
set.seed(20250102L)
DNS_Percolation(2500L, 2500L, 2000L, OBF_Type='C',
  OBF_iPara1=2, OBF_iPara2=-1, OBF_Ratio=0.3)
```

`FigLatticePlot` *Show the percolation cases*

Description

This function visualizes the state distribution at each lattice point on square, triangular, or hexagonal lattices. Nodes are typically classified into three categories: stable at "1", stable at "0", and oscillatory. The output of the function depends on the `ggplot` class.

Usage

```
FigLatticePlot(
  Values = NULL,
  Length = NULL,
  Height = NULL,
  Type = c(4L, 3L, 6L),
  ColorFills = c("#efefef", "#EE312E", "#2156A6"),
  LineColor = "#111111",
  Linewidth = 0.7
)
```

Arguments

`Values`	an IntegerVector, the results from DNS_Percolation
`Length`	an integer, the lngth of lattice
`Height`	an integer, the height of lattice
`Type`	an integer, 4=square; 3=triangle; 6=hexagon
`ColorFills`	a CharacterVector, [1] unstable, [2]0-stable, [3]1-stable; The string should conform to color syntax, such as "`red`" or "`#FF0000`".
`LineColor`	a string that describes the color of polygonal boundary. `NA` means the boundary is not displayed.
`Linewidth`	a numeric value, the line width of polygonal boundary.

Value

a `ggplot` object, show the percolation case.

`LoadBoolBioNetwork` *Load a Boolean genetic network*

Description

Load genetic network information from external files.

Usage

```
LoadBoolBioNetwork(
  NetName,
  NetType = c("BoolExpression", "TruthTable", "Threshold"),
  ThresMode = c("zero", "one", "self")
)
```

Arguments

`NetName` A string that denotes a valid file path or name for the target network.

`NetType` Which input format should be used? Options are restricted to `BoolExpression`, `TruthTable`, and `Threshold`.

- `BoolExpression`: All genes in a single file are represented as Boolean expressions.

```
X_1 = X_3 AND X_4
X_2 = NOT (X_1 OR X_3)
... ...
```

- `TruthTable`: All genes stored in a specified directory, organized according to the structural framework of Cell Collective Website.

```
in /your_path/NetworkName/X_1.csv (for gene X_1)
   X_3, X_5, X_1
    0 ,  0 ,  0
    0 ,  1 ,  1
    1 ,  0 ,  1
    1 ,  1 ,  0
in /your_path/NetworkName/X_2.csv (for gene X_2)
   X_4, X_2
    0 ,  0
    1 ,  1
    ... ...
```

- `Threshold`: Each line represents one regulatory interaction in a threshold network, where 1 denotes activation and 2 denotes inhibition.

```
X_1  X_3  1
X_4  X_2  2
X_5  X_2  1
 ... ...
```

`ThresMode` Threshold networks may exhibit cases where the sum of upstream inputs is zero, and different threshold patterns are defined as follows:

- `zero`: the controlled node is set to 0.
- `one`: the controlled node is set to 1.

- `self`: the controlled node retains its current state. In this case, each node is added a self-regulatory edge.

This parameter is effective only when the `NetType` parameter is `Threshold`.

Value

A four-element formatted list that records information of a Boolean network. (See BoolGRN_CellCollective)

`LoadManualNetwork` *Load a user-defined network*

Description

Instead of importing from files, this function enables users to define regulatory relationships and network structures to explore various system features.

Usage

```
LoadManualNetwork(
  Object,
  WhichCols = c(1, 2),
  BoolFunList = NULL,
  OBF_ratio = 0,
  OBF_type = "R"
)
```

Arguments

`Object`	A formatted data to record regulatory patterns; support `matrix`, `data.frame`, `igraph` object (if available).
`WhichCols`	An IntegerVector, specifies which columns should serve as the source and target columns (Default: `c(1,2)`).
`BoolFunList`	A list in which each element is a Boolean function, with the list length equal to the system size. If `NULL`, the function will generate a list of Boolean functions randomly based on the parameters `OBF_ratio` and `OBF_type`. (Default: `NULL`)
`OBF_ratio`	A numeric value in `[0,1]` representing the proportion of ordered Boolean function; this value is invalid if `BoolFunList` is `FALSE`.
`OBF_type`	A character denotes the type of ordered Boolean function, acceptable parameters are detailed in BoolFun_Generator.

Value

A four-element formatted list that records information of a Boolean network (See BoolGRN_CellCollective).

LoadMulValBioNets *Load a multi-valued genetic network*

Description

Load genetic network information from external files.

Usage

```
LoadMulValBioNets(NetName)
```

Arguments

NetName	A string that denotes a valid file path or name for the target network.

Value

A four-element formatted list that records information of a multi-valued network. Note that the 4-th elements save weights rather than the Boolean funciton.

MulV2Bool_Bool2MulV *Boolean to multi-valued and multi-valued to Boolean transformation*

Description

Boolean to multi-valued and multi-valued to Boolean transformation

Usage

```
MulV2Bool_Bool2MulV(
  aVec,
  k = 0,
  L = 3,
  Thres = NA,
  Bool2MulV = TRUE,
  MappingTable = FALSE
)
```

Arguments

aVec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system (Default: 3).
Thres	An IntegerVector containing thresholds for a multi-valued system. Its length is k+1, with each element ranging from 1 to L-1. If not provided, the function randomly generate them (Default: NA).
Bool2MulV	A logical value. Is the original function Boolean? If TRUE the transformation involves converting it from Boolean to multi-valued. Otherwise, from multi-valued to Boolean. (Default: TRUE).
MappingTable	A logical value. Is the mapping table also returned? (Default: FALSE)

Value

An IntegerVector (or IntegerMatrix if `MappingTable` is `TRUE`), denoting "mapping table" of target system.

Examples

```
# Transform a Boolean function to a multi-valued one.
# The Boolean is f(x,y)=x & y, a possible result see follows:
# set.seed(2025);
# MulV2Bool_Bool2MulV(c(0,0,0,1), k=2, L=3,
#   c(1,1,1), <--- the two inputs and one output share same threholds:
#                  in this case,  0 mapto 0 and 1 mapto {1,2}
#   MappingTable = TRUE);
# Return
#       v1 v0 xx   #  Boolean system
#  [1,]  0  0  0   #    0  0  0
#  [2,]  0  1  0   #    0  1  0
#  [3,]  0  2  0   #    0  1  0
#  [4,]  1  0  0   #    1  0  0
#  [5,]  1  1  2   #    1  1  1
#  [6,]  1  2  2   #    1  1  1
#  [7,]  2  0  0   #    1  0  0
#  [8,]  2  1  1   #    1  1  1
#  [9,]  2  2  2   #    1  1  1
set.seed(2025)
MulV2Bool_Bool2MulV(c(0,0,0,1), 2, 3, c(1,1,1), MappingTable=TRUE)
```

MulVFun_Complexity *Calculate the complexity of a multi-valued function*

Description

The analysis depends on average of prime implicants in multi- valued function and its complement(s), based on Quine-McCluskey method for multi-valued systems. Although this function is downward compatible with Boolean scenarios, it is recommended to use Boolean-specific counterpart (BoolFun_Complexity) directly due to its superior performance.

Usage

```
MulVFun_Complexity(aVec, k, L = 2)
```

Arguments

aVec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system (Default: 2).

Value

A numeric value representing the complexity of a multi-valued function.

Examples

```
# Calculate the complexity of a Booelan function (Show the compatibility).
# MulVFun_Complexity(c(0,0,1,0, 0,1,1,0), k=3)
# {(*10)+(101) ==> 1 (00*)+(*00)+(*11) ==> 0 } -> (2+3)=5
# Return 5
MulVFun_Complexity(c(0,0,1,0, 0,1,1,0), 3) # Compatible with Boolean system

# Calculate the complexity of a Ternary function.
# MulVFun_Complexity(c(1,1,1, 0,2,1, 1,0,2), k=2, L=3)
# { [(10)|(21)] ==> 0; [(0*)|(12)|(20)] ==> 1; [(11)|(22)] ==> 2} ->
# (2+3+2)=7
# Return 7
MulVFun_Complexity(c(1,1,1, 0,2,1, 1,0,2), k=2, L=3) # Ternary system
```

MulVFun_Effectiveness *Calculate the input/edge effetiveness of a multi-valued function*

Description

The analysis depends on multi-valued Quine-McCluskey method [Petrik,2008] to obtain the prime implicants of input vectors belong to sets "$f(x) = 0$", "$f(x) = 1$", ..., "$f(x) = i$", ..., "$f(x) = L - 1$", respectively.

Usage

```
MulVFun_Effectiveness(aVec, k, L = 2, Detail = FALSE, Redundancy = FALSE)
```

Arguments

aVec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system (Default: 2).
Detail	A logical value. Should return detail information of each edge? (Default: FALSE)
Redundancy	A logical value. Should return the corresponding redundant values of all edges rather than the effective ones? (Default: FALSE)

Details

The results can be obtained through either direct or indirect methods, which are equivalent in outcome. These computational processes are transparent to R users and do not impact usability. Advanced developers can utilize prototype functions to conduct various analyses based on specific requirements. Relevant concepts and definitions can be found in paper [Gates2021].

Value

A numeric value (or a NumericVector) denotes the effetive or redundant edge(s) of a multi-valued function.

Examples

```
# Calculate the effetive input of a ternary function c(0,1,2, 0,1,2, 0,1,2).
# Its can be transformed to 0=f(*0), 1=f(*1), 2=f(*2)
# MulVFun_Effectiveness(c(0,1,2, 0,1,2, 0,1,2), k=2, L=3)
# Return 1.000 (Only lower bit can control mapping table)
MulVFun_Effectiveness(c(0,1,2, 0,1,2, 0,1,2), k=2, L=3)

# Calculate the effetive edges of a ternary function c(0,1,2, 0,1,2, 2,2,2).
# Its can be transformed to 0=f(00)|f(10), 1=f(01)|f(11), 2=f(2*)|f(*2)
# MulVFun_Effectiveness(c(0,1,2, 0,1,2, 2,2,2), k=2, L=3, Detail=TRUE)
# Return 0.72222 0.72222
# Explain the values: 0.72222
#  00 | 01 | 02 | 10 | 11 | 12 | 20 | 21 | 22
#  00 | 01 | *2 | 10 | 11 | *2 |    |    | *2
#    |    |    |    |    |    | 2* | 2* | 2*
# ( 1 +  1 +  1 +  1 +  1 +  1 +  0 +  0 + 0.5 )/9 = 0.72222 # Lower one.
# (1  + 1  + 0  + 1  + 1  + 0  + 1  + 1  + 0.5 )/9 = 0.72222 # Higher one.
MulVFun_Effectiveness(c(0,1,2, 0,1,2, 2,2,2), k=2, L=3, Detail=TRUE)
```

`MulVFun_ExampleFuns` *three exemplified multi-valued functions*

Description

three exemplified multi-valued functions

Usage

```
data(MulVFun_ExampleFuns)
```

Details

They are saved as data.frame. Call the variable directly to see the details.

- `Example_3v_LogicalFun`: a three-valued logical function
- `Example_5v_LogicalFun`: a five-valued logical function
- `Example_3v_Multiplexer`: a three-valued Multiplexer

Detail information, please see original paper [Petrik2008].

`MulVFun_Generator` *Generate a specific type of multi-valued function*

Description

This function can generate following types: Canalization, Linear threshold-based, Dominated-valued.

Usage

```
MulVFun_Generator(
  MF_Type = c("R", "C", "D", "T"),
  k = 3L,
  L = 3L,
  Bias = NULL,
  CanaDeep = 1L,
  CanaVar = NULL,
  CanaVarNum = NULL,
  CanaLayerInfo = NULL,
  MappingTable = FALSE
)
```

Arguments

`MF_Type`	A character included in the following: • `C`: Canalization • `D`: Dominated-valued • `T`: Linear threshold-based • `R`: Random
`k`	An integer that denotes the number of input variables (Default: 3).
`L`	An integer that represents the level of multi-valued system (Default: 3).
`Bias`	A NumericVector representing the probabilities of elements in non-controlled slots within mapping tables (See `Details`). By default, `NULL`, which corresponds to the vector `rep(1.0/L, L)`. If provided, the vector must have a length greater than L-1 and contain only non-negative real numbers. The normalization of its first L elements determines the baseline configuration.
`CanaDeep`	An integer ranging from 1 to k, indicating k-layer canalization for type `'C'`. (Default: 1)
`CanaVar`	A `CanaDeep`-length non-repeating IntegerVector, indicating which variables should be canalized, with values ranging from 1 to k.
`CanaVarNum`	An integer vector of length `CanaDeep` specifying the number of each canalizing variable (ranging from 1 to L). If `CanaLayerInfo` is appropriately provided, `CanaVarNum` is ignored. (Default: `NULL`, automatically set using `sample.int(L, CanaDeep, replace=TRUE)` implemented in C++, rather than the native R function).
`CanaLayerInfo`	A list containing detail information for canalization: [[1]] for input (canalizing patterns); [[2]] for output (canalized patterns). This argument must meet specific requirements (see `Details`). If not provided (Default: `NULL`), the function will

automatically generate a random configuration. For detailed meaning, refer to BoolFun_Type.

`MappingTable` A logical value. Is the mapping table also returned? (Default: `FALSE`)

Details

The parameter Bias applies to non-controlled slots in the mapping table. For instance, a canalized function (k=3 and L=3) where $f(x_1 = 0) = 2$, indicates that only inputs with $x_1 \neq 0$ can be assigned probabilistic values according to the specified bias.

The `CanaLayerInfo` must satisfy the following requirements: it must contain at least two sub-lists, representing the canalizing and canalized configurations, respectively. Each sub-list must have a length exceeding `k`. Within each sub-list, all canalizing and canalized values must be strictly less than `L`. Furthermore, the elements in `CanaLayerInfo[[1]][[i]]` must be unique, which represent different canalizing values (See examples of MulVFun_is_NestedCana). Any violation of these conditions will lead to an error message and execution halts.

Value

An integervector of length L^k.

Examples

```
# Generate 3-input ternary function. Please note the canalizing/canalized
# values in the following scenarios:
set.seed(1234L)
Example_01=MulVFun_Generator('C', 3L, 3L, MappingTable=TRUE)
Example_02=MulVFun_Generator('T', 3L, 3L, MappingTable=TRUE)
Example_03=MulVFun_Generator('D', 3L, 3L, MappingTable=TRUE)
# 1=Yes, -1=No (for Canalized)
# 1=Yes, 0=Unknown, -1=No (for Threshold and Domainted)

MulVFun_is_NestedCana(Example_01[,4],3L,3L)[[1]]
# 1
MulVFun_is_Threshold(Example_01[,4],3L,3L)[[1]]
# -1
MulVFun_is_Domainted(Example_01[,4],3L,3L)[[1]]
# -1
MulVFun_is_NestedCana(Example_02[,4],3L,3L)[[1]]
# -1
MulVFun_is_Threshold(Example_02[,4],3L,3L)[[1]]
# 1
MulVFun_is_Domainted(Example_02[,4],3L,3L)[[1]]
# -1
MulVFun_is_NestedCana(Example_03[,4],3L,3L)[[1]]
# -1
MulVFun_is_Threshold(Example_03[,4],3L,3L)[[1]]
# -1
MulVFun_is_Domainted(Example_03[,4],3L,3L)[[1]]
# 1
```

`MulVFun_is_Domainted` *Decode the information of a multi-valued domaineted function*

Description

The function analyzes and returns a list containing information on whether the given function is domaineted and corresponding weights and baselines (deltas). It is also compatible with Boolean system (L=2).

Usage

```
MulVFun_is_Domainted(aVec, k, L, PrintOut = FALSE)
```

Arguments

`aVec`	An IntegerVector that represents a mapping table of multi-valued function.
`k`	An integer that denotes the number of input variables.
`L`	An integer that represents the level of multi-valued system.
`PrintOut`	A logical value. Should the output be displayed in the terminal?

Value

A list containing three elements:

- `[[1]] IsDomaint`: Is domainted? 1 for Yes, 0 for No.
- `[[2]] Weight`: The weights of variables, ordered from low to high bits.
- `[[3]] Delta`: The baselines of variables, ordered from low to high bits.

Examples

```
# Show the information of a multi-valued linear threshold function.
# set.seed(1002);
# A_MulV_Fun=MulVFun_Generator("D", k=2L, L=3L, MappingTable=TRUE);
# View(A_MulV_Fun); # Show the mapping table.
# A_MulV_Fun_Domaint=MulVFun_is_Domainted(A_MulV_Fun[,3], 2, 3, TRUE);
# Terminal output:
# f(~): {w_1=-5, w_0=3 | delta_0=-1, delta_1=0, delta_2=1}
# Above info means: avg max{i| \sum(w_i*x_i)+delta_i }
# A_MulV_Fun_Domaint # Show the information
# $IsDomaint
# [1] 1
# $Weight
# [1] 3 -5
# $Delta
# [1] -1  0  1

set.seed(1002)
A_MulV_Fun=MulVFun_Generator("D", k=2L, L=3L, MappingTable=TRUE)
A_MulV_Fun_Domaint=MulVFun_is_Domainted(A_MulV_Fun[,3], 2, 3, TRUE)
```

`MulVFun_is_NestedCana` *Analyze the structure of a multi-valued nested canalized function*

Description

This function decodes and visually represents the nested canalized structure (`data.frame`). Due to algorithm, the ID of the smaller variable is returned when variables at different levels share the same canalized values (See the second example). It is also compatible with the Boolean system, although `BoolFun_NestedCanalized` is recommended due to its binary characteristics.

Usage

```
MulVFun_is_NestedCana(aVec, k, L, PrintOut = FALSE)
```

Arguments

`aVec`	An IntegerVector that represents a mapping table of multi-valued function.
`k`	An integer that denotes the number of input variables.
`L`	An integer that represents the level of multi-valued system.
`PrintOut`	A logical value. Should output be displayed in the terminal? (Default: `FALSE`).

Value

A list containing two elements:

- `[[1]]` an `integer`: is canalized? (1 for Yes, 0 for No).
- `[[2]]` a `dataframe`: the nested structure of a canalized function.

Examples

```
# Show the canalized structure of a multi-valued nested canalized function.
# Return a data.frame (Also show nested "if-else-if" hierarchical relations).
# set.seed(2025);
# A_MulV_Fun=MulVFun_Generator("C", k=4L, L=3L, CanaDeep=3,
#   CanaVar=c(2,1,3), # Order is 2nd, 1st, 3nd.
#   CanaLayerInfo=list(
#     list(c(2,0),c(0),c(1,2)),  # Canalizing values of each variable
#     list(c(0,1),c(2),c(2,0))), # Canalized values of each variable
#   MappingTable=TRUE);
# View(A_MulV_Fun);# Show the mapping table.
# A_MulV_Fun_Neseted=MulVFun_is_NestedCana(A_MulV_Fun[,5], 4, 3, TRUE);
# Terminal output:
# if:
# x_1=0 ==> f(x)=1
# x_1=2 ==> f(x)=0
# else if:
#  ..., x_0=0 ==> f(x)=2
# else if:
#  ...,  ..., x_2=1 ==> f(x)=2
#  ...,  ..., x_2=2 ==> f(x)=0
# View(A_MulV_Fun_Neseted); # Show the nested structure.
```

```
set.seed(2025);
A_MulV_Fun=MulVFun_Generator("C", k=4L, L=3L, CanaDeep=3,
  CanaVar=c(2,1,3), CanaLayerInfo=list(list(c(2,0),c(0),c(1,2)),
  list(c(0,1),c(2),c(2,0))), MappingTable=TRUE);
A_MulV_Fun_Neseted=MulVFun_is_NestedCana(A_MulV_Fun[,5], 4, 3, TRUE);

# Second example for same level issue.
# set.seed(2020);
# A_MulV_Fun=MulVFun_Generator("C", k=4L, L=2L, CanaDeep=2,
#   CanaVar=c(2,3), CanaLayerInfo=list(list(c(0),c(1)), list(c(1),c(1))),
#   MappingTable = TRUE);
# Please compare the mapping table and printed information.
# A_MulV_Fun_Neseted=MulVFun_is_NestedCana(A_MulV_Fun[,5],4,2,TRUE);
# if:
# x_0=1 ==> f(x)=1
# else if:
#  ..., x_1=1 ==> f(x)=1
# else if:
#  ...,  ..., x_2=0 ==> f(x)=1
# else if:
#  ...,  ...,  ..., x_3=0 ==> f(x)=0
#  ...,  ...,  ..., x_3=1 ==> f(x)=1
# In fact, here set the first layer is f(x_1=0) ==> f(x)=1,
# it is also coupled with second layer f(x_0=0) ==> f(x)=1
set.seed(2020)
A_MulV_Fun=MulVFun_Generator("C", k=4L, L=2L, CanaDeep=2, CanaVar=c(2,3),
  CanaLayerInfo=list(list(c(0),c(1)), list(c(1),c(1))), MappingTable=TRUE)
A_MulV_Fun_Neseted=MulVFun_is_NestedCana(A_MulV_Fun[,5],4,2,TRUE)
```

`MulVFun_is_Signed` *Decode the information of a multi-valued signed function*

Description

The function analyzes and returns a list containing information whether given function is signed, and corresponding weight matrix, and baselines. It is also compatible with Boolean system (L=2).

Usage

```
MulVFun_is_Signed(aVec, k, L, PrintOut = TRUE)
```

Arguments

`aVec`	An IntegerVector that represents a mapping table of multi-valued function.
`k`	An integer that denotes the number of input variables.
`L`	An integer that represents the level of multi-valued system.
`PrintOut`	A logical value. Should output be displayed in the terminal? (Default: `TRUE`).

Value

A list containing three elements:

- [[1]] IsSigned: Is Signed? 1 for Yes, 0 for No.
- [[2]] Weight: The weights of all pairwise relationships among varibales; it is a L*(kL) matrix.
- [[3]] Delta: The baselines of variables, ordered from low to high bits.

Examples

```
# Generate a domainted function and analyze it belong to signed class or not.
# set.seed(1001);
# A_MulV_Fun=MulVFun_Generator("D", k=3L, L=3L, MappingTable=TRUE);
# A_MulV_Fun_Sign=MulVFun_is_Signed(A_MulV_Fun[,4], 3, 3, TRUE);
# View(A_MulV_Fun); # Show the mapping table
#       v2 v1 v0 f_out
#  [1,]  0  0  0     2
#  [2,]  0  0  1     2
#  [3,]  0  0  2     1
#  [4,]  0  1  0     2
#  [5,]  0  1  1     0
#  [6,]  0  1  2     0
#          ... ...
# Check the "signed" regulatory patterns (User can test other cases)
# A_MulV_Fun_Sign[[2]]%*%c(1,0,0, 1,0,0, 1,0,0) ==> (-4,-1,0) ==> f_out(0,0,0)=2;
# A_MulV_Fun_Sign[[2]]%*%c(0,0,1, 1,0,0, 1,0,0) ==> (-2,-1,-5) ==> f_out(0,0,2)=1;
# A_MulV_Fun_Sign[[2]]%*%c(0,1,0, 0,0,1, 1,0,0) ==> (4,-1,1) ==> f_out(0,2,1)=0;
# ... ...

set.seed(1001)
A_MulV_Fun=MulVFun_Generator("D", k=3L, L=3L, MappingTable=TRUE)
A_MulV_Fun_Sign=MulVFun_is_Signed(A_MulV_Fun[,4], 3, 3, TRUE)
A_MulV_Fun_Sign[[2]]%*%c(1,0,0, 1,0,0, 1,0,0)
A_MulV_Fun_Sign[[2]]%*%c(0,0,1, 1,0,0, 1,0,0)
A_MulV_Fun_Sign[[2]]%*%c(0,1,0, 0,0,1, 1,0,0)
```

MulVFun_is_Threshold *Decode the information of a multi-valued linear threshold function*

Description

This function can return a list records information of whether given function is threshold, and corresponding weights and boundaries. It is also compatible with Boolean system (L=2).

Usage

```
MulVFun_is_Threshold(aVec, k, L, PrintOut = FALSE)
```

Arguments

aVec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system.
PrintOut	A logical value. Should output be displayed in the terminal? (Default: TRUE).

Value

A list containing three elements:

- [[1]] IsThreshold: Is threshold based? 1 for Yes, 0 for No.
- [[2]] Weight: The weight of each variable.
- [[3]] Boundary: Records the L intervals; the L-1 boundaries, ordered from low to high bits (See examples).

Examples

```
# Show the information of a multi-valued linear threshold function.
# set.seed(2025);
# A_MulV_Fun=MulVFun_Generator("T", k=3L, L=3L, MappingTable=TRUE);
# View(A_MulV_Fun) # Show the mapping table
# A_MulV_Fun_Thres=MulVFun_is_Threshold(A_MulV_Fun[,4], 3, 3, TRUE);
# Return:
# f(~)=2*x_2-1*x_1-2*x_0
# Threshold intervals for L=3 values: (-inf,-3], (-3,0], (0,+inf)

set.seed(2025)
A_MulV_Fun=MulVFun_Generator("T", k=3L, L=3L, MappingTable=TRUE)
A_MulV_Fun_Thres=MulVFun_is_Threshold(A_MulV_Fun[,4], 3, 3, TRUE)

# Single input Quaternary funciton:
# A_MulV_Fun_Thres=MulVFun_is_Threshold(c(1,1,2,3), 1, 4, TRUE);
# Return:
# f(~)=1*x_0
# Threshold intervals for L values: (-inf,-1], (-1,1], (1,2], (2,+inf)

A_MulV_Fun_Thres=MulVFun_is_Threshold(c(1,1,2,3), 1, 4, TRUE)
```

MulVFun_Polynomial *Convert a multi-valued function as polynomial like forms*

Description

Unlike the Boolean system, we adopt a definition similar to that of domaineted function; please refer to MulVFun_is_Domainted.

Usage

```
MulVFun_Polynomial(aVec, k = 2, L = 3)
```

Arguments

aVec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables (Default: 2).
L	An integer that represents the level of multi-valued system (Default: 3).

Value

A list containing three elements:

- [[1]] sat: Is satisfiability? TRUE or FALSE
- [[2]] Wights: The weights and thresholds.
- [[3]] HighOrder: The highest order of terms contained in the function (See examples).

Examples

```
# Convert the a multi-valued function into a polynomial like expression.
# In fact, the result is same as MulVFun_is_Domainted().
# MulVFun_Polynomial(c(0,1,2,1,1,2,2,1,2), k=2L, L=3L);
# Return
# [[1]] sat:TRUE,
# [[2]] Weight:
#    x_1         x_2 threshold_1 threshold_2 threshold_3
#      2           1          -2           0           0
# [[3]] HighOrder: 1

MulVFun_Polynomial(c(0,1,2,1,1,2,2,1,2), k=2L, L=3L)
```

MulVFun_QMForm *Show the Quine-McCluskey form of a multi-valued function*

Description

The function employs our native C++ standard multi-valued Quine-McCluskey algorithm to obtain prime implicants. Because of algorithms and system properties, the result is not unique for multi-valued system. See reference [Petrik2008].

Usage

```
MulVFun_QMForm(
  aVec,
  k,
  L,
  VarsName = NULL,
  ShowType = c("print", "data.frame"),
  SourceTable = FALSE
)
```

Arguments

aVec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system.
VarsName	A CharacterVector that represents each name of variable.
ShowType	Which output format should be used? Options are restricted to `print` (printed in terminal) and `data.frame` (in data.frame form).
SourceTable	A logical value. Should return the source mapping table? It is ignored when ShowType is `print` (Default: FALSE).

Value

Three different cases:

- NULL: Only print in terminal (ShowType=="print")
- data.frame: ShowType is "data.frame" and SourceTable is FALSE.
- list: ShowType is "data.frame" and SourceTable is TRUE. [[1]] is the result; [[2]] is the source funciton.

Examples

```
# Calculate the Quine-McCluskey form of multi-valued function.
# Analyze a three-valued multiplexer:
#             | a, iff x=0
# f(x,c,b,a)= | b, iff x=1
#             | c, iff x=2
# Example_3v_Multiplexer; # Show the multiplexer.
# MulVFun_QMForm(Example_3v_Multiplexer[,5], k=4, L=3,
#   VarsName=colnames(Example_3v_Multiplexer)[4:1]);
#
# Return its QMC form:
# x  c  b  a f(*)
# 2  1 NA NA    1
# 2  2 NA NA    2
# 1 NA  1 NA    1
# 1 NA  2 NA    2
# 0 NA NA  1    1
# 0 NA NA  2    2
MulVFun_QMForm(Example_3v_Multiplexer[,5], k=4, L=3,
   VarsName=colnames(Example_3v_Multiplexer)[4:1])
```

MulVFun_Recovery *Recover a multi-valued function from multi-valued model*

Description

Unlike the standard multi-valued logical paradigm, we adopt the definition of so-called qualitative networks; please refer to [Schaub2007]

Usage

```
MulVFun_Recovery(aMulVNet, GeneName, MaxValue = 2L, MapTab = FALSE)
```

Arguments

aMulVNet	A four-element formatted list that records information of a multi-valued
GeneName	A string that denotes the gene name.
MaxValue	An integer that represents the max value of the gene (Default: 2).
MapTab	A logical value. Is the mapping table also returned? (Default: FALSE)

Value

An IntegerVector or a mapping table.

MulVFun_Sensitivity *Calculate the sensitivity of a multi-valued function*

Description

Correspodning concepts of multi-valued systems are generalized from the Boolean system.

Usage

```
MulVFun_Sensitivity(Map_Vec, k, L)
```

Arguments

Map_Vec	An IntegerVector that represents a mapping table of multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system.

Value

A numeric value representing the sensitivity of the given function.

Examples

```
# Show the canalized structure of a multi-valued nested canalized function.
# MulVFun_Sensitivity(c(0,1,2, 1,0,1, 2,2,0), k=2, L=3);
# Return 1.777778

MulVFun_Sensitivity(c(0,1,2, 1,0,1, 2,2,0), k=2, L=3)
```

r_CheckValidBoolFun *Boolean function checker*

Description

Check whether the input vector is a valid Boolean function

Usage

```
r_CheckValidBoolFun(Bit_Vec)
```

Arguments

Bit_Vec	A Boolean vector (or 0/1-valued IntegerVector) represents the truth table of a Boolean function.

Value

a List: [[1]] Boolean vector, [[2]] in-degree

r_CheckValidMulVFun *Multi-valued function checker*

Description

Check whether the input vector is a valid multi-valued function

Usage

```
r_CheckValidMulVFun(a_Vec, k, L)
```

Arguments

a_Vec	An IntegerVector representing a multi-valued function.
k	An integer that denotes the number of input variables.
L	An integer that represents the level of multi-valued system.

Value

IntegerVector or prints an error message and execution halts.

r_GenerateTruthTable *Generate the header of truth table*

Description

Generate state enumeration table for the given number of input variables

Usage

```
r_GenerateTruthTable(inDegree = 2L, SpinLike = FALSE)
```

Arguments

inDegree	An integer that denotes the number of input variables.
SpinLike	A logical value. Should output the spin-like form?

Value

A data.frame of input case enumeration

r_GenerateTruthTable_M *Generate the header of mapping table*

Description

Generate state enumeration table for the given number of input variables (multi-valued version)

Usage

```
r_GenerateTruthTable_M(inDegree = 2L, Lsystem = 2L)
```

Arguments

inDegree	An integer that denotes the number of input variables (Default: 2).
Lsystem	An integer that represents the level of multi-valued system (Default: 2).

Value

a data.frame of input case enumeration

r_UnfreezeInputNode *Add self-loop circuits to all external nodes*

Description

Convert all input and external nodes into self-loop nodes

Usage

```
r_UnfreezeInputNode(RealBioNet)
```

Arguments

RealBioNet	A four-element formatted list that records information of a Boolean network (See BoolGRN_CellCollective)

Value

A four-element formatted list that records information of the modified Boolean network (See BoolGRN_CellCollective)

Transient_Process_A *Simulating roughly attractor analysis*

Description

Like DNS_DamageSpread, but record final state.

Usage

```
Transient_Process_A(
  RealBioNet,
  SampleNum = 100L,
  ExtSelfLoop = TRUE,
  Controller = NULL,
  ConVals = NULL,
  ExternalNode = NULL,
  Times = 1000L,
  NumSys = 2L,
  UpdateRule = 1L
)
```

Arguments

RealBioNet	A four-element formatted list that records information of a Boolean network.
SampleNum	An integer pointing the number of randomly sampling initial states.
ExtSelfLoop	A logical value. Should inputs be converted into self-loops? (Default: TRUE)
Controller	An integer or character vector indicating the nodes to be controlled. (Default: NULL)

ConVals	A Boolean vector specifying the values of controlled of controlled nodes. (Default: NULL)
ExternalNode	A Boolean vector specifying the states of non-input nodes. (Default: NULL, indicating random assignment)
Times	an integer, steps of simulating system.
NumSys	an integer, number of discrete value.
UpdateRule	an integer, update rule: 1=synchronous, 2=asynchronous, 3=quick-asynchronous.

Details

See DNS_DamageSpread.

Value

A matrix where each row represents the final state of a stochastic simulation, and each column represents a gene.

Transient_Process_G *Simulating a transient process (Automatically generate network)*

Description

Same as DNS_DamageSpread, but record the transient process.

Usage

```
Transient_Process_G(
  Size = 1000L,
  SimStep = 1000L,
  Init_Dist = 0.1,
  Init_1_Ratio = c(0.5, 0.5),
  OBF_Type = "R",
  OBF_iPara1 = 1L,
  OBF_iPara2 = 1L,
  OBF_Ratio = 0.1,
  RBF_Bias = c(0.5, 0.5),
  Net_Type = "K",
  Net_fPara = 4,
  NumSys = 2L,
  UpdateRule = 1L
)
```

Arguments

Size	an integer, size of system.
SimStep	an integer, steps of simulating system.
Init_Dist	a numeric value, initial normalized Hamming distance of two vector.
Init_1_Ratio	a NumvericVector, proportion of each value (its length is NumSys).
OBF_Type	a character, ordered Boolean function type (OBF).

OBF_iPara1	an integer, configuration parameter for OBF.
OBF_iPara2	an integer, configuration parameter for OBF (Not necessary for some types of OBF).
OBF_Ratio	a numeric value, proportion of ordered Boolean function within system.
RBF_Bias	a NumvericVector, biases of random function (each is (0,1), sum=1).
Net_Type	a character, system topological type.
Net_fPara	a numeric value, topological configured parameters.
NumSys	an integer, number of discrete value.
UpdateRule	an integer, update rule: 1=synchronous, 2=asynchronous, 3=quick-asynchronous.

Details

See DNS_DamageSpread.

Value

A list containing two elements:

- [[1]] Transient states: A matrix whose rows and columns represent time points and nodes, respectively.
- [[2]] Network: The structure is consistent with that in BoolGRN_CellCollective

---

Transient_Process_L *Simulating a transient process (Loaded network)*

Description

Same as DNS_DamageSpread, but record the transient process.

Usage

```
Transient_Process_L(
  RealBioNet,
  ExtSelfLoop = TRUE,
  Controller = NULL,
  ConVals = NULL,
  ExternalNode = NULL,
  Times = 1000L,
  Init_1_Ratio = c(0.5, 0.5),
  NumSys = 2L,
  UpdateRule = 1L
)
```

Arguments

`RealBioNet`	A four-element formatted list that records information of a Boolean network.
`ExtSelfLoop`	A logical value. Should inputs be converted into self-loops? (Default: `TRUE`)
`Controller`	An integer or character vector indicating the nodes to be controlled. (Default: `NULL`)
`ConVals`	A Boolean vector specifying the values of controlled of controlled nodes. (Default: `NULL`)
`ExternalNode`	A Boolean vector specifying the states of non-input nodes. (Default: `NULL`, indicating random assignment)
`Times`	an integer, steps of simulating system.
`Init_1_Ratio`	a NumvericVector, proportion of each value (its length is `NumSys`).
`NumSys`	an integer, number of discrete value.
`UpdateRule`	an integer, update rule: 1=synchronous, 2=asynchronous, 3=quick-asynchronous.

Details

See DNS_DamageSpread.

Value

A list containing two elements:

- `[[1]]` `Transient states`: A matrix whose rows and columns represent time points and nodes, respectively.
- `[[2]]` `Network`: The structure is consistent with that in BoolGRN_CellCollective

---

`Transient_Process_M` *Simulating roughly attractor analysis of multi-valued model*

Description

Like Transient_Process_A, but use multi-valued model.

Usage

```
Transient_Process_M(RealBioNet_M, SampleNum = 100L, Times = 1000L, NumSys = 2L)
```

Arguments

`RealBioNet_M`	A four-element formatted list that records information of a multi-valued threshold network.
`SampleNum`	An integer pointing the number of randomly sampling initial states.
`Times`	an integer, steps of simulating system.
`NumSys`	an integer, number of discrete value.

Details

See the reference [Schaub2007]

Value

A matrix where each row represents the final state of a stochastic simulation, and each column represents a gene.

Index